

Kernel Support Vector Classifiers (K-SVC)



Anubha Gupta, PhD.
Professor
SBILab, Dept. of ECE,
IIIT-Delhi, India
Contact: anubha@iiitd.ac.in; Lab: <http://sbilab.iiitd.edu.in>



INDRAPRASTHA INSTITUTE *of*
INFORMATION TECHNOLOGY **DELHI**

The NPTEL logo is a circular emblem. It features a central stylized flower or star shape with eight petals, rendered in a light orange color. This central motif is surrounded by a ring of smaller, repeating petal-like shapes in a light pink color. The entire circular logo is set against a background of a light blue gradient.

Kernel Support Vector Classifiers (K-SVC)

NPTEL

Learning Objectives

In this module, we will learn the following:

- Support Vector Classifiers (SVC): Recap
- Terminology of Support Vector Machines
- Motivation of Kernel SVM
- Kernel Substitution
- Kernel SVC
- Mercer's Condition: Valid Kernel
- Popular Kernels
- Multiclass Kernel SVC
- Support Vector Regression (SVR)

NPTEL

Support Vector Classifier (SVC): Recap

- Hard Margin Support Vector Classifier Dual Formulation:

s.t.

$$\min_{\eta_i, \eta_j} \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \eta_i \eta_j y_i y_j (\mathbf{x}_i^T \mathbf{x}_j)$$

Dot Product

$$\sum_{j=1}^n \eta_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j + b \geq 1 \quad \forall i$$

$$\underline{x}_i^T \underline{x}_j = \langle \underline{x}_i, \underline{x}_j \rangle$$

feature vector of i^{th} training sample

feature vector of j^{th} training sample

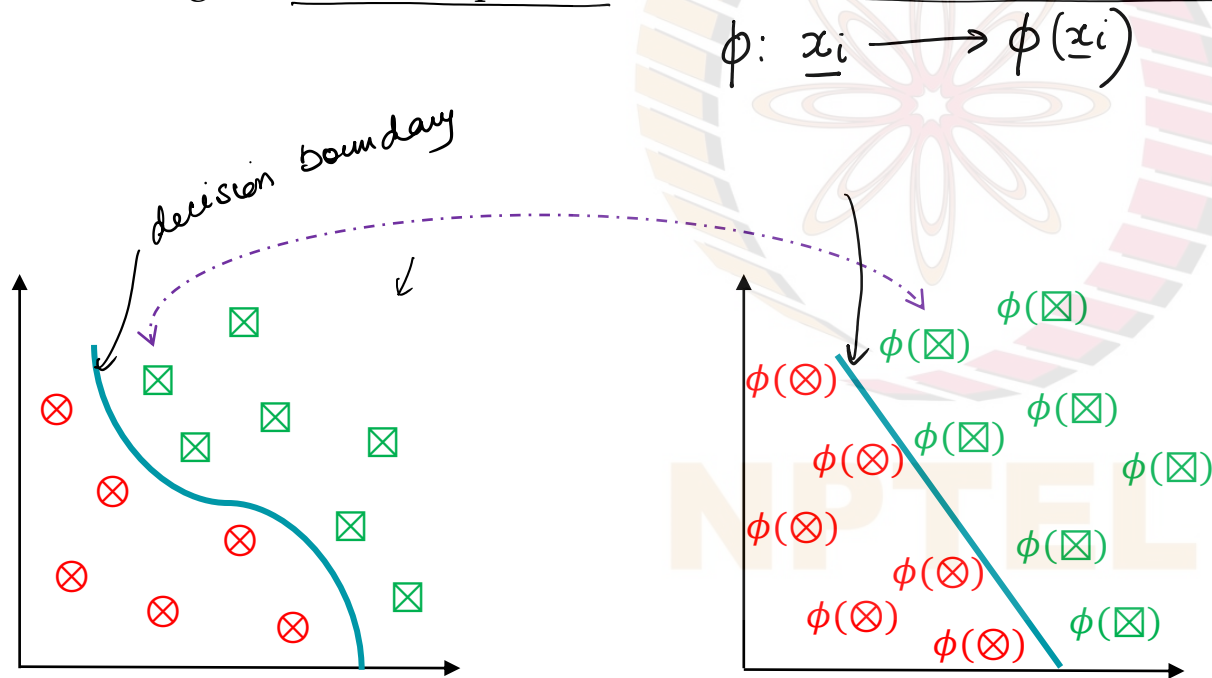
Terminology

- The terms SVC and SVM are often used interchangeably
- Below are the terminological descriptions
 - **Support Vector Classifiers (SVC)**: Maximum margin classifiers for linear data
 - **Support Vector Machines (SVM)** or **Kernel SVC** or **Kernel Machines**: An extension of SVC that use kernels to alter the feature space*
 - **Support Vector Regression (SVR)**: An extension of SVM to accommodate regression problems

NPTEL

Motivation

- Limitation of linear classifiers, such as the standard SVC, cannot perform well with intrinsically non-linear data
- Enlarge the feature space to accommodate a non-linear boundary between classes



Kernel Substitution

- Define the kernel function as

$$\mathcal{K}(\mathbf{x}_i, \mathbf{x}_j) = \Phi(\mathbf{x}_i)^T \Phi(\mathbf{x}_j)$$

- In the context of SVC, kernel function $\mathcal{K}(\cdot)$ computes the similarity between data samples in the higher dimensional space
- Kernel Substitution or Kernel Trick:** One can simply obtain the value of the dot product (or the inner product) by applying the **Kernel Function** without getting to know the actual values of individual samples after **Kernelization**
- In other words, we do not require to explicitly compute $\Phi(\mathbf{x}_j)$, but are required to compute directly the $\mathcal{K}(\mathbf{x}_i, \mathbf{x}_j)$
- Many different kernel functions may correspond to the same inner product
- Gram Matrix:** Matrix of kernel values $\mathcal{K}(\cdot)$.

n train sample $\phi: \underline{x}_i \rightarrow \phi(\underline{x}_i)$
 Gram matrix $= K =_{n \times n}$

$$\begin{bmatrix}
 \mathcal{K}(\underline{x}_1, \underline{x}_1) & \mathcal{K}(\underline{x}_1, \underline{x}_2) & \dots \\
 \vdots & \vdots & \ddots \\
 \mathcal{K}(\underline{x}_n, \underline{x}_1) & \dots & \mathcal{K}(\underline{x}_n, \underline{x}_n)
 \end{bmatrix}$$

Kernel SVC

- An extension of the SVC algorithm for nonlinear data using Kernel Trick
- Allows for non-linear, complex decision boundaries
- Replace dot product of $\mathbf{x}_i^T \mathbf{x}_j$ with $\Phi(\mathbf{x}_i)^T \Phi(\mathbf{x}_j)$

s.t.

$$\min_{\eta_i, \eta_j} \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \eta_i \eta_j y_i y_j (\Phi(\mathbf{x}_i)^T \Phi(\mathbf{x}_j))$$

$$\sum_{j=1}^n \eta_j y_i y_j \Phi(\mathbf{x}_i)^T \Phi(\mathbf{x}_j) + b \geq 1 \quad \forall i$$

NPTEL

Example

- House will be sold or not

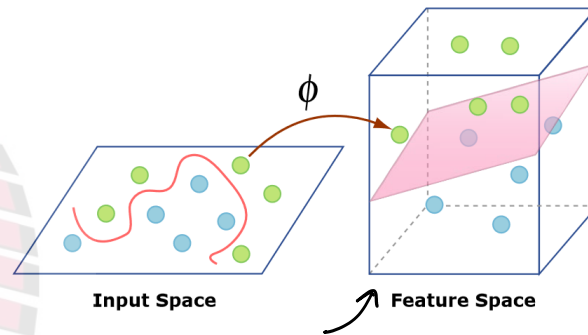
$$\mathbf{x}_i = \begin{bmatrix} x_{i1} \\ x_{i2} \end{bmatrix}$$

[Area, Distance from marketplace]

$$\phi: \mathbb{R}^2 \rightarrow \mathbb{R}^3$$

$$\mathbf{x}_i = \begin{bmatrix} x_{i1} \\ x_{i2} \end{bmatrix} \rightarrow \Phi(\mathbf{x}_i) = \begin{bmatrix} x_{i1} \\ x_{i2} \\ x_{i1}x_{i2} \end{bmatrix}$$

- The 2D space gets mapped to a 3D space.
- This modified feature space is known as the **Reproducing Kernel Hilbert Space (RKHS)**



https://ocw.mit.edu/courses/15-097-prediction-machine-learning-and-statistics-spring-2012/resources/mit15_097s12_lec13/

NPTEL

Mercer's Condition: Valid Kernel

- For a function $\mathcal{K}$ to be a valid kernel, the two necessary and sufficient conditions are:
 - Symmetry:** Symmetric to its arguments

$$\mathcal{K}(\mathbf{x}_1, \mathbf{x}_2) = \mathcal{K}(\mathbf{x}_2, \mathbf{x}_1)$$

- Positive (Semi) Definiteness**

$$\mathbf{z}^T \mathcal{K} \mathbf{z} \geq 0 \quad \forall \text{ non-zero } \mathbf{z}$$

Such a kernel is said to satisfy the Mercer's condition.

Summary of Kernel Trick

"For any **valid kernel**, there exists a corresponding mapping function. However, it is often simpler to calculate $\mathcal{K}(\mathbf{x}_i, \mathbf{x}_j)$ than calculating the individual mappings $\Phi(\mathbf{x}_i)$ & $\Phi(\mathbf{x}_j)$ and computing their dot product."

Popular Kernels

- **Polynomial Kernel:**

$$\mathcal{K}(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^T \mathbf{x}_j + r)^d$$

- d: degree of polynomial
- r: offset

- **Linear Kernel:**

$$\mathcal{K}(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^T \mathbf{x}_j + 1)^1$$

- Polynomial kernel with $d = r = 1$

- **Sigmoid Kernel:**

$$\mathcal{K}(\mathbf{x}_i, \mathbf{x}_j) = \tanh(2\mathbf{x}_i^T \mathbf{x}_j + 1)$$

- $\tanh(\cdot)$ has the same shape as sigmoid except that its range is $[-1, 1]$

Popular Kernels

- Distance Kernel:

$$\mathcal{K}(\mathbf{x}_i, \mathbf{x}_j) = \exp \left[-\frac{\mathcal{D}(\mathbf{x}_i, \mathbf{x}_j)}{2\sigma^2} \right]$$

$e^{-\text{large}} \rightarrow \text{small}$

- Radial Basis Kernel:

$$\mathcal{K}(\mathbf{x}_i, \mathbf{x}_j) = \exp \left[-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma^2} \right] \leftarrow$$

- It is a spherical kernel with $\mathbf{x}_i$ as center and σ is the radius
- Distance kernel with Euclidian distance as distance metric

- Mahalanobis Kernel:

$$\mathcal{K}(\mathbf{x}_i, \mathbf{x}_j) = \exp \left[-\frac{1}{2} (\mathbf{x}_i - \mathbf{x}_j)^T \mathbf{K}^{-1} (\mathbf{x}_i - \mathbf{x}_j) \right]$$

- $\mathbf{K}$ is a covariance matrix
- Distance kernel with Mahalanobis distance as distance metric

Note: It is possible to define application-specific customized kernels. Beyond the current scope.

Let us Try!

Question: Show that for the polynomial kernel function $\mathcal{K}: \mathbb{R}^2 \times \mathbb{R}^2 \rightarrow \mathbb{R}$ defined as

$$\mathcal{K}(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^T \mathbf{x}_j + 1)^2$$

can be expressed as

$$\mathcal{K}(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$$

where $\phi: \mathbb{R}^2 \rightarrow \mathbb{R}^6$ is defined by:

$$\underline{x}_i = \begin{bmatrix} x_{i1} \\ x_{i2} \end{bmatrix}$$

$$\phi(\mathbf{x}_i) = \begin{pmatrix} 1 \\ \sqrt{2}x_{i1} \\ \sqrt{2}x_{i2} \\ x_{i1}^2 \\ x_{i2}^2 \\ \sqrt{2}x_{i1}x_{i2} \end{pmatrix}$$

You may pause the video and try.

Let us Try!

Answer:

$$\boxed{\mathcal{K}(\mathbf{x}_i, \mathbf{x}_j) = (\mathbf{x}_i^T \mathbf{x}_j + 1)^2} = (x_{i1}x_{j1} + x_{i2}x_{j2} + 1)^2$$

$$\mathcal{K}(\mathbf{x}_i, \mathbf{x}_j) = 1 + 2x_{i1}x_{j1} + 2x_{i2}x_{j2} + x_{i1}^2x_{j1}^2 + x_{i2}^2x_{j2}^2 + 2x_{i1}x_{i2}x_{j1}x_{j2}$$

$$= \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j) = \begin{pmatrix} 1 & \sqrt{2}x_{i1} & \sqrt{2}x_{i2} & x_{i1}^2 & x_{i2}^2 & \sqrt{2}x_{i1}x_{i2} \end{pmatrix} \begin{pmatrix} 1 \\ \sqrt{2}x_{j1} \\ \sqrt{2}x_{j2} \\ x_{j1}^2 \\ x_{j2}^2 \\ \sqrt{2}x_{j1}x_{j2} \end{pmatrix}$$

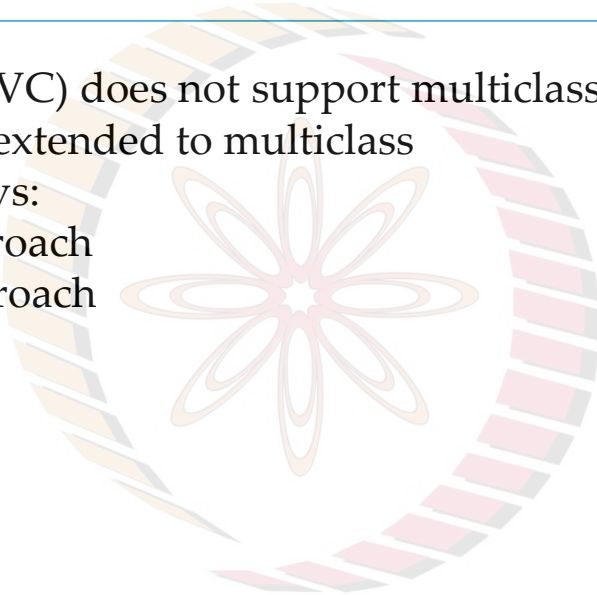
$\uparrow$ 1×6

6×1

Hence, the polynomial kernel defined above corresponds to the inner product of the data samples transformed using ϕ in the higher dimensional feature space, and thus quantify the similarity of the data samples in the feature space.

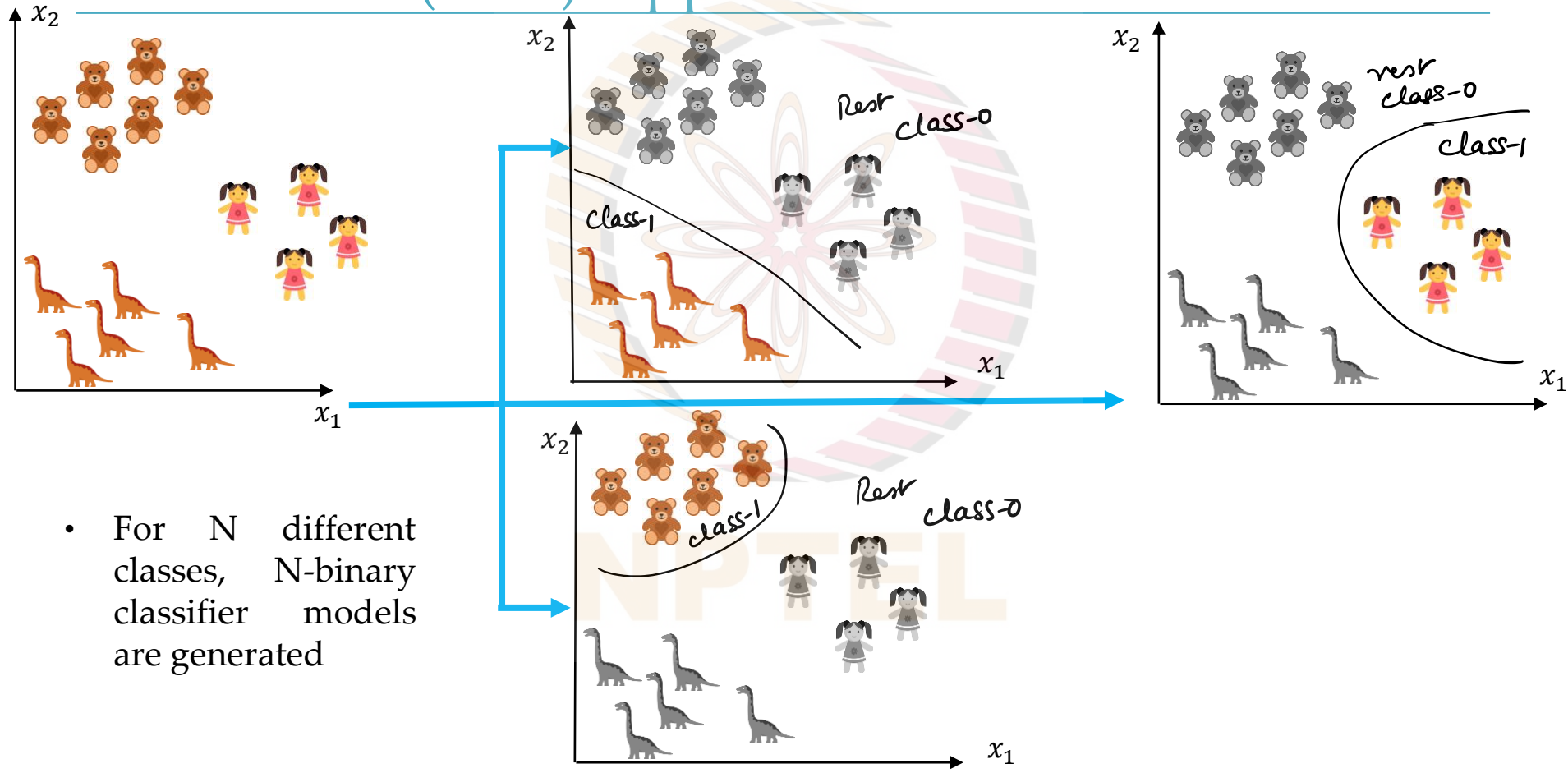
Multiclass SVC

- Support Vector Classifier (SVC) does not support multiclass classification natively.
- Binary classification can be extended to multiclass
- This can be done in two ways:
 1. **One-vs-Rest (OvR)** Approach
 2. **One-vs-One (OvO)** Approach

The NPTEL logo is a circular emblem. It features a central stylized flower or star shape with eight petals. Surrounding this center is a ring composed of many small, rectangular segments in shades of orange, yellow, and pink. Below the emblem, the word "NPTEL" is written in large, bold, orange capital letters.

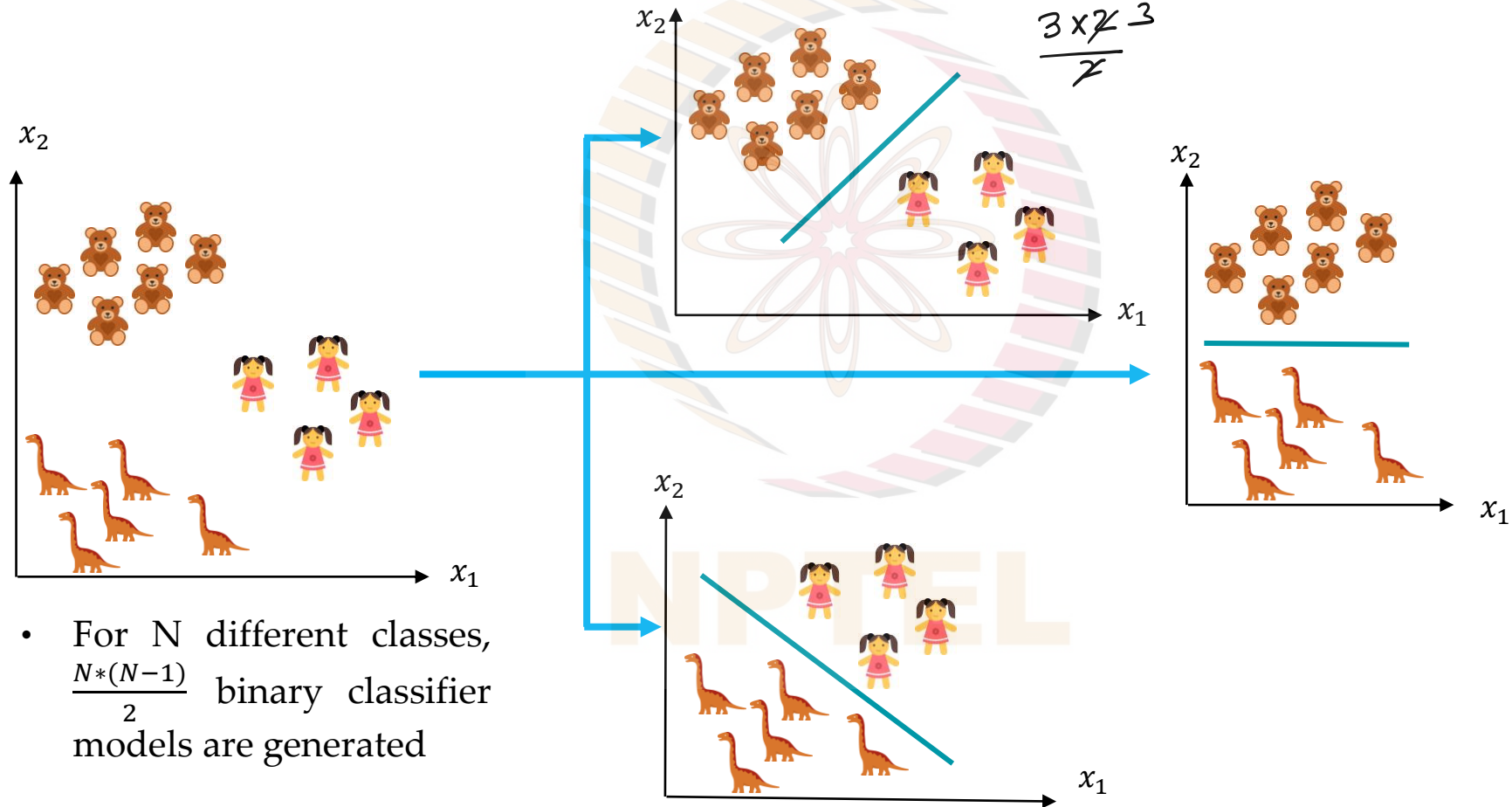
NPTEL

One-vs-Rest (OvR) Approach



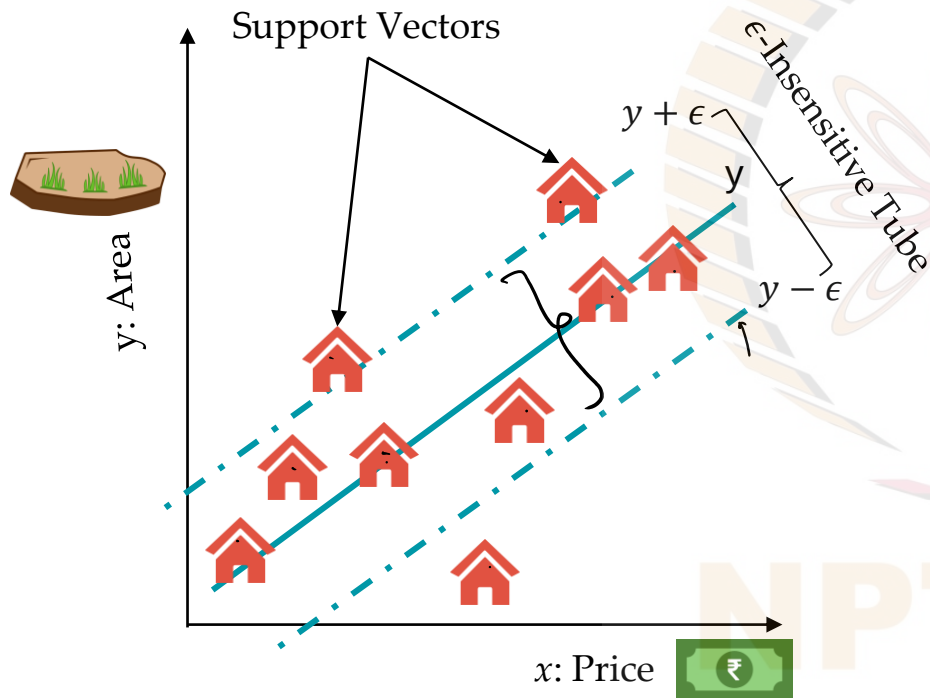
- For N different classes, N -binary classifier models are generated

One-vs-One (OvO) Approach



- For N different classes, $\frac{N*(N-1)}{2}$ binary classifier models are generated

Support Vector Regression (SVR)



- An extension of SVM to accommodate regression problems
- Seeks to find a hyperplane that best fits the data points in a continuous space while maximizing the margin (distance) between the closest data points (support vectors)
- Error is calculated with reference to the samples outside the insensitive tube, known as support vectors supported by

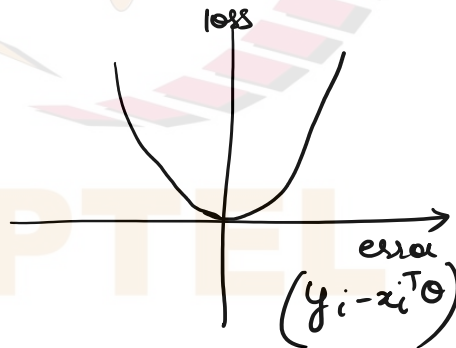
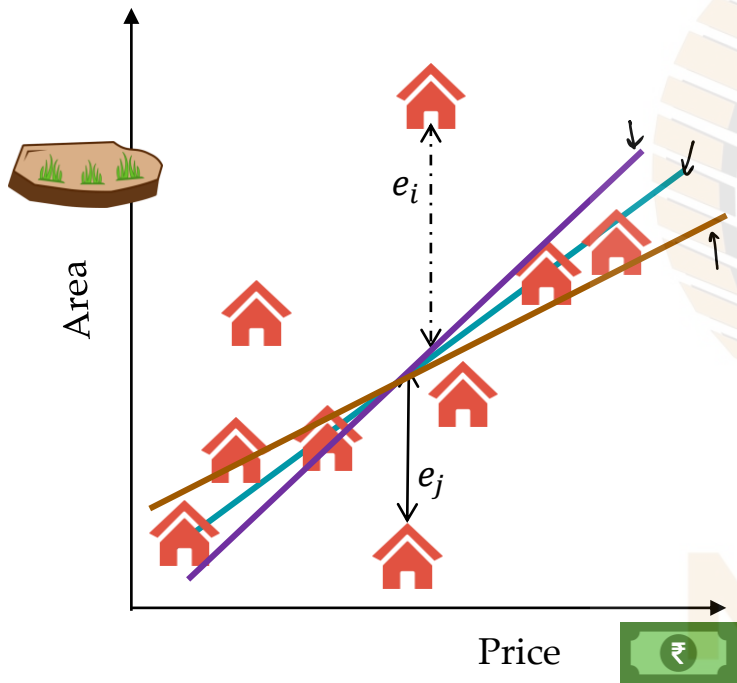
Linear Regression Revisited

Linear Regression: Which of the fitted lines is the best?

The loss function is defined as the sum of squared errors-

$$\sum_{i=1}^n (y_i - \mathbf{x}_i^T \boldsymbol{\theta})^2$$

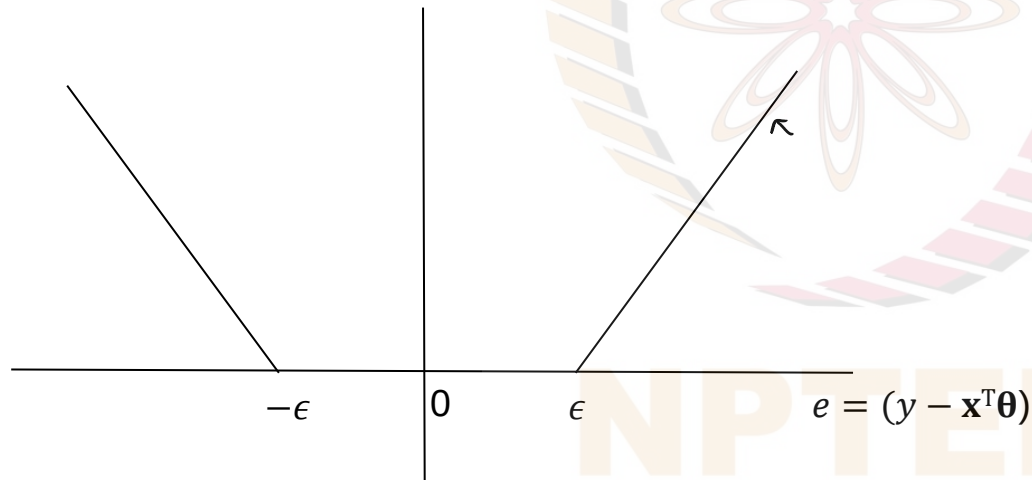
This loss grows quadratically with error.



Support Vector Regression (SVR)

- SVR uses an ϵ -insensitive linear loss function shown as below-

$$Loss = \begin{cases} 0 & \text{if } |y - \mathbf{x}^T \boldsymbol{\theta}| < \epsilon \\ |y - \mathbf{x}^T \boldsymbol{\theta}| - \epsilon & \text{otherwise} \end{cases}$$



Support Vector Regression (SVR)

Given: a data set $\{\mathbf{x}_1, \dots, \mathbf{x}_n\}$ with target values $\{y_1, \dots, y_n\}$, we want to build a ϵ -SVR.

The optimization problem is

$$\min_{\theta} \left[\frac{1}{2} \|\theta\|^2 + C \sum_{i=1}^n (\xi_i + \xi_i^*) \right] \quad \text{subject to} \begin{cases} y_i - \theta^T \mathbf{x}_i - b \leq \epsilon + \xi_i \\ \theta^T \mathbf{x}_i + b - y_i \leq \epsilon + \xi_i^* \\ \xi_i \geq 0, \xi_i^* \geq 0 \end{cases}$$

which is solved using quadratic programming.



To Summarize

In this module, we learned:

- About the Kernel SVC
- Popular Kernels
- Multiclass Kernel SVC
- Support Vector Regression (SVR)

